DATA REPRESENTATION

Bit: this stands for binary digit. It is the smallest unit of data in a computer. A bit has a single binary value, either 0 or 1.

Nibble: a collection of four bits. This is useful in representing hexadecimal numbers.

Byte: A byte is a collection of eight bits and is the smallest addressable data item.

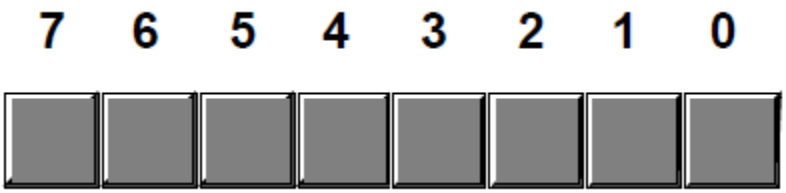


Figure 1: Bit numbering in a byte

Bit 0 is the *low order bit* or *least significant bit* and bit 7 is the *high order bit* or *most significant bit* of the byte.

Bits 0..3 comprise the *low order nibble*, bits 4..7 form the *high order nibble*. Since a byte contains exactly two nibbles.

Since a byte contains eight bits, it can represent 2^8, or 256, different values. Generally, we'll use a byte to represent numeric values in the range 0..255, signed numbers in the range -128..+127, ASCII character codes, and other special data types requiring no more than 256 different values. Many data types have fewer than 256 items so eight bits is usually sufficient.

Probably the most important use for a byte is holding a character code. Characters typed at the keyboard; displayed on the screen, and printed on the printer all have numeric values.

Word: A word is a group of 16 bits. The bits in a word are numbered starting from zero on up to fifteen. Like the byte, bit 0 is the low order bit and bit 15 is the high order bit. When referencing the other bits in a word their bit position number is used. Notice that a word contains exactly two bytes. Bits 0 through 7 form the low order byte, bits 8 through 15 form the high order byte. With 16 bits, you can represent 2^{16} (65,536) different values. These could be the values in the range 0 to 65,535 (or, as is usually the case, -32,768..+32,767) or any other data type with no more than 65,536 values.

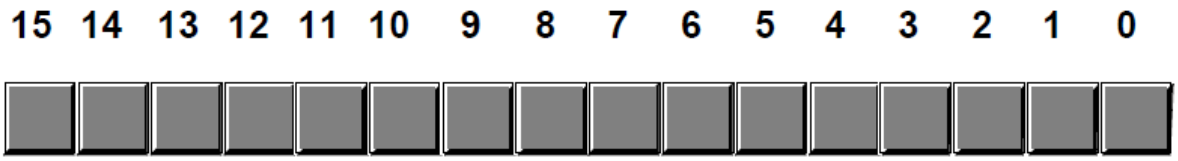


Figure 2: Bit numbers in a word.

Double word

A double word is a pair of words. Therefore, a double word quantity is 32 bits long.

Naturally, this double word can be divided into a high order word and a low order word, or four different bytes, or eight different nibbles.

Double words can represent all kinds of different things. First and foremost on the list is a segmented address. Another common item represented with a double word is a 32-bit integer value.



Figure 3: Bit numbering in a double word

CHARACTER CODES

Human-understandable characters must be converted to computer-understandable bit patterns using some sort of character encoding scheme. The following are some of those schemes:

Binary Coded Decimal (BCD)

Binary-coded Decimal (BCD) is a numeric coding system used primarily in IBM mainframe and midrange systems in the 1950s and 1960s. It is very common in electronics, particularly those that display numerical data, such as alarm clocks and calculators. BCD encodes each digit of a decimal number into a 4-bit binary form. When stored in an 8-bit byte, the upper nibble is called the **zone** and the lower part is called the **digit**.

Digit	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
Zones	
1111	Unsigned
1100	Positive
1101	Negative

Advantages of BCD Codes

- It is very similar to decimal system.
- We need to remember binary equivalent of decimal numbers 0 to 9 only.

Disadvantages of BCD Codes

- The BCD arithmetic is a little more complicated: the addition and subtraction of BCD have different rules.
- BCD needs more number of bits than binary to represent the decimal number. So BCD is less efficient than binary.

EBCDIC

The Extended Binary-Coded Decimal Interchange Code (EBCDIC) was an extension of the BCD. It was mainly used in IBM mainframe computers. In an EBCDIC file, each alphabetic or numeric

character is represented with an 8-bit binary number (a string of eight 0's or 1's). 256 possible characters (letters of the alphabet, numerals, and special characters) are defined.

ASCII

The American Standard Code for Information Interchange code is divided into four groups of 32 characters. The first 32 characters, ASCII codes 0 through 1Fh (31), form a special set of non-printing characters called the **control characters**. They perform various printer/display control operations rather than displaying symbols. Examples include *carriage return*, which positions the cursor to the left side of the current line of characters8, *line feed* (which moves the cursor down one line on the output device), and *back space* (which moves the cursor back one position to the left). However, there is no standardization as different control characters perform different operations on different output devices.

The second group of 32 ASCII character codes comprises various **punctuation symbols**, **special characters**, and the **numeric digits**. The most notable characters in this group include the space character (ASCII code 20h) and the numeric digits (ASCII codes 30h..39h). Note that the numeric digits differ from their numeric values only in the H.O. nibble. By subtracting 30h from the ASCII code for any particular digit you can obtain the numeric equivalent of that digit.

The third group of 32 ASCII characters is reserved for the **upper case alphabetic characters**.
The ASCII codes for the characters “A”…”Z” lie in the range 41h...5Ah (65...90). Since there are only 26 different alphabetic characters, the remaining six codes hold various **special symbols**.

The fourth, and final, groups of 32 ASCII character codes are reserved for the **lower case alphabetic symbols**, five additional **special symbols**, and another **control character (delete)**. Note that the lower case character symbols use the ASCII codes 61h…7Ah. If you convert the codes for the upper and lower case characters to binary, you will notice that the upper case symbols differ from their lower case equivalents in exactly one bit position.
For example, consider the character code for “E” and “e”.
The only place these two codes differ is in bit five. Upper case characters always contain a zero in bit five; lower case alphabetic characters always contain a one in bit five. You can use this fact to quickly convert between upper and lower case. If you have an uppercase character you

can force it to lower case by setting bit five to one. If you have a lower case character and you wish to force it to upper case, you can do so by setting bit five to zero. You can toggle an alphabetic character between upper and lower case by simply inverting bit five.

Indeed, bits five and six determine which of the four groups in the ASCII character set you're in:

Bit 6	Bit 5	Group
0	0	Control Characters
0	1	Digits & Punctuation
1	0	Upper Case & Special
1	1	Lower Case & Special

So you could, for instance, convert any upper or lower case (or corresponding special) character to its equivalent control character by setting bits five and six to zero.

Consider, for a moment, the ASCII codes of the numeric digit characters:

"0" 48 30h
"1" 49 31h
"2" 50 32h
"3" 51 33h
"4" 52 34h
"5" 53 35h
"6" 54 36h
"7" 55 37h
"8" 56 38h
"9" 57 39h

The decimal representations of these ASCII codes are not very enlightening. However, the hexadecimal representation of these ASCII codes reveals something very important – the L.O. nibble of the ASCII code is the binary equivalent of the represented number. By stripping away (i.e., setting to zero) the H.O. nibble of a numeric character, you can convert that character code to the corresponding binary representation. Conversely, you can convert a binary value in the range 0...9 to its ASCII character representation by simply setting the H.O. nibble to three. Note that you can use the logical-AND operation to force the H.O. bits to zero; likewise, you can use the logical-OR operation to force the H.O. bits to 0011 (three).

Note that you *cannot* convert a string of numeric characters to their equivalent binary representation by simply stripping the H.O. nibble from each digit in the string. Converting

123 (31h 32h 33h) in this fashion yields three bytes: 010203h, not the correct value which is 7Bh. Converting a string of digits to an integer requires more sophistication than this; the conversion above works only for single digits.

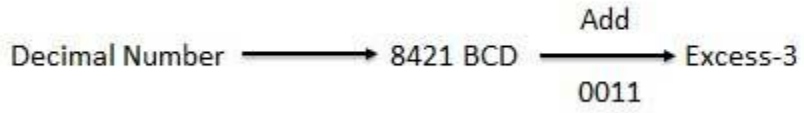
Bit seven in standard ASCII is always zero. This means that the ASCII character set consumes only half of the possible character codes in an eight bit byte. IBM uses the remaining 128 character codes for various special characters including international characters (those with accents, etc.), math symbols, and line drawing characters. Note that these extra characters are a non-standard extension to the ASCII character set.

UNICODE

Both EBCDIC and ASCII were built around the Latin alphabet. In 1991, a new international information exchange code called Unicode was developed. Unicode is a **16-bit** alphabet that is downward compatible with ASCII and Latin-1 character set. Because the base coding of Unicode is 16 bits, it has the capacity to encode the majority of characters used in every language of the world.

Excess-3 code

The Excess-3 code is also called XS-3 code. It is a non-weighted code used to express decimal numbers. The Excess-3 code words are derived from the BCD code words adding $(0011)_2$ or $(3)_{10}$ to each code word. The excess-3 codes are obtained as follows −



Decimal	BCD 8 4 2 1	Excess-3 BCD + 0011
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

Gray Code

It is a non-weighted code and is not an arithmetic code. That means there are no specific weights assigned to the bit position. It has a very special feature that, only one bit will change each time the decimal number is incremented as shown in the figure below. Since only one bit changes at a time, the gray code is also called a unit distance code. The gray code is a cyclic code. Gray code cannot be used for arithmetic operation.

Decimal	BCD	Gray
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1

Application of Gray code

- Gray code is popularly used in the shaft position encoders.
- A shaft position encoder produces a code word which represents the angular position of the shaft.